# Manuel Utilisateur

Compilateur Deca – Version 1.0

Équipe GL53

Janvier 2026

## Table des matières

**1 Introduction** **2**

**2 Utilisation de base** **2**
2.1 Invocation du compilateur . . . . . 2
2.2 Options disponibles . . . . . 2
2.3 Sortie du compilateur . . . . . 2

**3 Limitations de cette implémentation** **2**
3.1 Limitations générales . . . . . 2
3.2 Limitations du langage supporté . . . . . 2
3.3 Effets pour l'utilisateur . . . . . 3

**4 Messages d'erreur** **3**
4.1 Erreurs lexicales . . . . . 3
4.2 Erreurs syntaxiques . . . . . 3
4.3 Erreurs contextuelles (sémantiques) . . . . . 3
4.4 Erreurs d'exécution (runtime IMA) . . . . . 4

**5 Diagnostics et résolution de problèmes** **4**
5.1 Le compilateur ne trouve pas decac . . . . . 4
5.2 Fichier .ass non généré . . . . . 4
5.3 Erreur « OutOfMemoryError » lors de la compilation . . . . . 4
5.4 Code exécuté incorrect mais pas d'erreur . . . . . 4

**6 Conformité et limites de test** **4**
6.1 Tests validés . . . . . 4
6.2 Cas non testés systématiquement . . . . . 5

**7 Support et contact** **5**

**8 Annexes** **5**
8.1 Résumé des options . . . . . 5
8.2 Codes de retour . . . . . 5
8.3 Variables d'environnement . . . . . 5

# 1 Introduction

Ce manuel s'adresse aux utilisateurs du compilateur Deca développé par l'équipe GL53. Il suppose que vous connaissez déjà les spécifications détaillées du langage Deca. Ce manuel décrit les fonctionnalités, limitations et extensions propres à cette implémentation.

# 2 Utilisation de base

## 2.1 Invocation du compilateur

```
decac [options] <fichier.deca>...
```

**Exemple :**

```
decac hello.deca           # Compilation simple
decac -p test.deca         # D compilation uniquement
decac -v -n prog.deca      # V rification sans contr les runtime
decac file1.deca file2.deca file3.deca  # Compilation multiple
```

## 2.2 Options disponibles

| Option | Description |
|---|---|
| `-b` | Affiche une bannière indiquant le nom de l'équipe. |
| `-p` | Mode décompilation : arrête après construction de l'AST et affiche le programme décompilé. |
| `-v` | Mode vérification : arrête après la phase de vérification contextuelle (pas de génération .ass). |
| `-n` | No-check : supprime les contrôles de débordement à l'exécution (stack overflow, division par zéro, etc.). Améliore les performances au détriment de la sécurité. |
| `-r X` | Limite le nombre de registres disponibles à X (4 X 16). Utile pour tester le compilateur avec des ressources limitées. |
| `-d` | Active les traces de debug. Peut être répété pour augmenter la verbosité. |
| `-P` | **Non implémenté** : compilation parallèle des fichiers. |
| `-w` | Active les avertissements (warnings). |

## 2.3 Sortie du compilateur

Pour chaque fichier `programme.deca`, le compilateur génère :
— `programme.ass` : code assembleur IMA (si pas d'erreur).
— Messages sur la sortie d'erreur standard en cas de problème.
Le fichier .ass est généré **au même endroit** que le fichier source.

# 3 Limitations de cette implémentation

## 3.1 Limitations générales

— **Compilation parallèle** : l'option `-P` n'est pas implémentée. Tous les fichiers sont compilés séquentiellement.
— **Optimisations** : le compilateur ne réalise aucune optimisation du code généré (pas de propagation de constantes, pas d'élimination de code mort, pas d'optimisation de boucles).
— **Gestion mémoire limitée** : pour les programmes très volumineux ($>$ 10000 lignes), des lenteurs peuvent survenir lors de l'analyse.

## 3.2 Limitations du langage supporté

— **Classes imbriquées** : non supportées. Une classe doit être déclarée au niveau racine du programme.
— **Surcharge de méthodes** : non supportée. Deux méthodes dans une même classe ne peuvent pas avoir le même nom.

— **Interfaces** : non supportées (pas dans la spécification de base).
— **Génériques** : non supportés.
— **Packages** : pas de système de packages ou modules.
— **Exceptions** : pas de mécanisme try/catch/throw.

### 3.3 Effets pour l'utilisateur

— Si vous déclarez une classe à l'intérieur d'une autre, le compilateur retournera une erreur syntaxique.
— Si vous tentez de définir deux méthodes avec le même nom dans une classe (même avec des signatures différentes), seule la première sera prise en compte ou une erreur sera levée.
— Les programmes très longs peuvent nécessiter plusieurs secondes de compilation.

## 4 Messages d'erreur

Tous les messages d'erreur suivent le format standard :

`<fichier>:<ligne>:<colonne>: error (<type>): <message>`

### 4.1 Erreurs lexicales

**Format :** `fichier.deca:ligne:colonne: error (lexical): message`

| Configuration | Message typique |
|---|---|
| Chaîne non fermée | `chaîne incomplète` ou `unterminated string` |
| Nombre malformé | `nombre invalide` ou `invalid number format` |
| Caractère illégal | `caractère invalide : '\uXXXX'` |
| Commentaire non fermé | `commentaire non terminé` |

**Exemple :**

```
{
    println("texte sans guillemet fermant);
}
```

**Erreur :** `test.deca:2:10: error (lexical): chaîne incomplète`

### 4.2 Erreurs syntaxiques

**Format :** `fichier.deca:ligne:colonne: error (syntax): message`

| Configuration | Message typique |
|---|---|
| Parenthèse/accolade manquante | `attendu ')' mais trouvé ''` |
| Point-virgule oublié | `attendu ';' mais trouvé <token>` |
| Instruction incomplète | `fin de fichier inattendue` |
| Mot-clé mal placé | `'else' sans 'if' correspondant` |

**Exemple :**

```
{
    println(x
}
```

**Erreur :** `test.deca:3:1: error (syntax): attendu ')' mais trouvé '}'`

### 4.3 Erreurs contextuelles (sémantiques)

**Format :** `fichier.deca:ligne:colonne: error (contextual): message`
**Exemple :**

```
{
    println(x);  // x non d clar e
}
```

**Erreur :** `test.deca:2:13: error (contextual): Identificateur 'x' non déclaré`

| Configuration | Message typique |
|---|---|
| Variable non déclarée | `Identificateur 'x' non déclaré` |
| Type incompatible | `Type incompatible : attendu int, trouvé float` |
| Méthode inexistante | `Méthode 'foo' introuvable dans la classe 'A'` |
| Classe non déclarée | `Classe 'B' non déclarée` |
| Double déclaration | `Variable 'x' déjà déclarée` |
| Cast invalide | `Cast impossible de int vers ClasseX` |
| Condition non booléenne | `Condition doit être de type booléen non pas int` |

### 4.4 Erreurs d'exécution (runtime IMA)

Ces erreurs surviennent lors de l'exécution du code assembleur .ass sur la machine virtuelle IMA.

| Configuration | Effet |
|---|---|
| Division par zéro | IMA affiche `Error: division by zero` |
| Débordement pile | IMA affiche `Error: stack overflow` |
| Accès mémoire invalide | IMA affiche `Error: segmentation fault` |
| Débordement arithmétique | IMA affiche `Error: overflow` (si `-n` non utilisé) |

**Note :** avec l'option `-n`, les contrôles de débordement sont désactivés, ce qui peut conduire à des résultats incorrects silencieux plutôt qu'à des erreurs.

## 5 Diagnostics et résolution de problèmes

### 5.1 Le compilateur ne trouve pas decac

**Cause :** le script `decac` n'est pas dans votre PATH ou n'a pas les permissions d'exécution.
**Solution :**

```
# Ajouter au PATH (sous Linux/Mac)
export PATH=/chemin/vers/gl53/src/main/bin:$PATH

# Donner permissions d'ex cution
chmod +x /chemin/vers/gl53/src/main/bin/decac
```

### 5.2 Fichier .ass non généré

**Causes possibles :**
— Erreurs de compilation (lexicales, syntaxiques, contextuelles).
— Option `-p` ou `-v` utilisée (arrêt avant génération de code).
**Solution :** vérifiez les messages d'erreur sur stderr et corrigez le programme source.

### 5.3 Erreur « OutOfMemoryError » lors de la compilation

**Cause :** programme source trop volumineux ou compilation de nombreux fichiers.
**Solution :** augmentez la mémoire allouée à la JVM en modifiant le script `decac` :

```
exec java -Xmx2G -enableassertions -cp "$CP" fr.ensimag.deca.DecacMain "$@"
```

### 5.4 Code exécuté incorrect mais pas d'erreur

**Cause probable :** option `-n` utilisée, désactivant les contrôles de débordement.
**Solution :** recompiler sans `-n` pour activer tous les contrôles runtime.

## 6 Conformité et limites de test

### 6.1 Tests validés

Le compilateur a été testé sur :

— Suite de tests lexicaux/syntaxiques (basic-lex.sh, basic-synt.sh).
— Suite de tests contextuels (basic-context.sh).
— Suite de tests de génération de code (basic-gencode.sh).
— Pipeline complet (common-tests.sh).

### 6.2 Cas non testés systématiquement

— Programmes avec héritage multi-niveaux ($> 3$ niveaux).
— Classes avec $> 50$ méthodes ou attributs.
— Boucles imbriquées $> 5$ niveaux de profondeur.
— Fichiers source $> 10000$ lignes.

## 7 Support et contact

Pour toute question ou problème non couvert par ce manuel :
— Consultez la documentation de validation (`docs/VALIDATION_COMPLETE.pdf`).
— Consultez le dépôt Git : `https://gitlab.ensimag.fr/.../gl53`.
— Contactez l'équipe GL53.

## 8 Annexes

### 8.1 Résumé des options

```
decac [[-p | -v] [-n] [-r X] [-d]* [-P] <fichier.deca>...] | [-b]
```

### 8.2 Codes de retour

— `0` : compilation réussie sans erreur.
— `1` : erreur de compilation (lexicale, syntaxique, contextuelle) ou erreur système.

### 8.3 Variables d'environnement

Aucune variable d'environnement spécifique n'est requise, hormis `JAVA_HOME` pour l'exécution de la JVM.